

{EPITECH}

VIROLOGY

< HANDLE WITH CARE />



VIROLOGY

You've been hired by an offensive security company.

Your team specializes in covert intrusions designed to test how quickly the Security Operations Center (SOC) of a large organization detects and responds to an attack.

Your first assignment, as a student acting like a professional, is to design and build a new offensive intrusion tool named *sOP0wn3d*.

This is a real build project: by the end, your team must demonstrate a working tool in your lab. Everything you implement, you must also be able to explain, justify, and discuss from both the attacker's and the defender's point of view.



This is a pedagogical project.

You **MUST NOT** carry out any kind of attack against a target without express and written consent, in accordance with the laws applicable to all parties.



Breaking the law can cost you both money and years of your life. Violating a company's trust can damage your reputation and jeopardize both current and future employment opportunities.

THIS NO JOKE!

Mission

Stealth is the foremost quality of your tool: *sOP0wn3d* must remain inconspicuous to monitoring solutions and to the naked eye. Intrusions can also last a long time, so it must be as **resilient** as possible.

The tool acts as a Command and Control (C2) system: the central hub from which you monitor, control, and interact with the machines compromised during an intrusion.

Studying existing exploitation tools and C2 frameworks (such as Metasploit, Mimikatz, TheFatRat, Sliver, Empire, etc.) is a good idea. But use them as architectural references only, to understand how they work, in particular the implant-to-controller design pattern.

Do not reuse them as your solution. You are expected to build your own tools from scratch.

Mandatory core

Your tool must, at a minimum:

- ✓ Run unobtrusively and remain undetected by the antivirus running in your declared lab environment.
- ✓ Spawn a remote shell on the target.
- ✓ Create a tunnel between the attacker and the target using an existing protocol (HTTP, DNS, IRC, SSH, FTP, RDP, and similar).
- ✓ Extract credentials from the compromised machine.
- ✓ Survive a reboot, and restart even if its process is stopped.
- ✓ Encrypt its communications, preferably with an asymmetric method, and let you prove it with a traffic analyzer such as Wireshark.

Discretion is essential.

You must be able to cover your tracks in the system, network, and command logs (by rolling them back rather than wiping everything), and to demonstrate that you did.



Extended capabilities

Once *sOP0wn3d* proves stealthy, resilient, and effective on its core, enrich it with remote commands. Each command below is a build target; the description states intent and the detection surface to reason about, not a recipe.

Command	Intent
keylog	Start, stop, or dump a capture of keyboard input.
rdp	Enable or disable the Remote Desktop service on the target.
crack	Launch password-hash cracking against material recovered from the target.
pth	Reuse a recovered hash to authenticate to another machine (pass-the-hash).
loot	Locate and exfiltrate sensitive files (password stores, SSH keys, configuration files).
phish	Assist in producing a credential-harvesting lure from the compromised context.
propagate	Move laterally toward other machines on the network.
privesc	Identify and exploit a local misconfiguration or vulnerability to elevate privileges.
syscall	Invoke a system call directly from userland to bypass userland API hooking.
shell	Offer a shell relying on built-in commands to avoid spawning new processes.
whatever	Be creative and surprise us.

Feel free to add parameters to your commands if you deem it necessary.

Rules and constraints



You may use low-level libraries (cryptography, networking, UI, and similar). You may not use any all-in-one tool that automates the job for you (for example Metasploit, Empire, Sliver, or Puppet). Build your own implementation from scratch, or modify existing exploits to make them yours.

Reaching for a popular utility (for credential access, scanning, cracking, and so on) is fast, but widely used code carries known signatures, which makes your tool easier to detect.

1. In a first step, you may use a known tool or library to get a capability working and to save time.
2. You document this as a known weakness of your tool, and you explain why it is detectable.
3. In a second step, you replace it with a stealthier implementation of your own.

This resolves the apparent contradiction between "do not rely on detectable third-party code" and "demonstrate that you can integrate real tooling": you do both, in that order, and you reason about the trade-off.

Your lab environment



You must provide a working tool and test it at least on a Windows virtual machine.

You choose and master your own environment. Select the target operating system and version yourself (a recent Windows is a sensible default, but the choice is yours), provision a disposable virtual machine, and snapshot it.

You own the demonstration: pick an environment you control fully so that your keynote runs smoothly, without the risk of a last-minute OS patch breaking your tool. Use an environment manifest and make your demo reproducible.

Deliverables

- ✓ A working tool demonstrated live on your lab virtual machine.
- ✓ A clear README for users, containing at least: how to install, how to use, external resources, and sources. Feature tutorials are a welcome bonus.
- ✓ For each implemented capability, a mapping to its MITRE ATT&CK tactic/technique ID and a short "blue team note".
- ✓ A clean, well-organized delivery: all required files present, no useless files (temporary, binary, or test artifacts), logically arranged.
- ✓ Maintainable code, free of obvious defects (duplication, functions doing several things at once, unclear responsibilities, dead variables).
- ✓ A clean version-control history: at least three branches and a coherent gitflow, with every member committing at least twice.
- ✓ A short presentation supported by accurate slides that complete your talk rather than replace it.



This project rewards the journey as much as the destination. You are expected to explore, get it wrong, adjust, and improve. A team that followed a sound method and reasoned well about its choices has learned something valuable even where a capability did not fully work. Be ready to argue your decisions and to answer questions with concrete, factual examples.

v 1.3

{EPITECH}